Changelog

Newest first. Hand-written. Update when a set of files is copied onto the USB, not on every save.

1.2.19: 2026-10-05

After the seed copy, the host builds a one-time **data card** from non-script documents (csv/txt/json/md, not PROMPT.md). Phase 0 gets that card with the request. The manager may write `test_plan` from those cell values; then the plan is locked. Coder `main.py` and tester literals must match the card (CSV numbers stay strings). Later rounds do not re-read the files.

1.2.18: 2026-10-05

`bench.py` skips frozen tasks (http-kv shipped on Qwen2.5 Coder 7B). Snapshot under `frozen/http-kv/`. `--tasks http-kv` will not re-run it.

1.2.17: 2026-10-05

Tool examples use `<FULL_SOURCE>` instead of `"..."`. A `sandbox_write` whose content is still that placeholder or `...` is rejected (`placeholder_content`); the full file text must go in `args.content`, not a markdown fence.

1.2.16: 2026-10-05

Coder cannot read, write, or execute `test_*.py`. Each impl write is compile-checked and then run (import lib or `python main.py`). Coder status hides test files. Tester execute of a path that is not on disk returns `missing_test` and asks for a write first (no longer treated as a missing SCORES print).

1.2.15: 2026-10-05

Host coding rules: do not give a field or variable the same name as a function or method it must call (`self.store = {}` hides `def store`). Tester job says write the test file first and only execute a path that is already on disk.

1.2.14: 2026-10-03

Host coding rules: a Python `'...'` / `"..."` string must stay on one physical line (use `/n`, triple quotes, or a data file). The csv-sql seed ships extra `data/*.csv` tables and tells the committee to query those files only, not CSV text built in Python.

1.2.13: 2026-10-03

Coder smoke imports library scripts, then runs `main.py` (it must exit 0). The inventory card lists calls found in `main.py`, including class methods. Tester tests must clone those calls with other literals; the host rejects a test that does not compile or that hardcodes a `SCORES metric=1,1,1` list. Quality is still not the coder's job at smoke time.

1.2.12: 2026-10-03

Coder and tester jobs are short host rules: coder writes impl only and must keep locked names; tester must write then execute and print n trial `SCORES` values (not a total). Last stderr, live inventory, and `live_name_required` sit at the top of those user turns so the seed prompt does not bury them. The host rejects a coder write that drops locked names, will not re-execute a `test_broken` script until it is rewritten, and skips auto-execute of a missing path.

1.2.11: 2026-10-02

While Coding is on, **Folder...** picks a local project folder (Windows and Linux file dialog). The host copies allowed files into a new `outputs/` work folder; the original is not edited. GitHub URL and a local folder cannot be used together. `TASK.md` / `PROMPT.md` / `README.md` in that folder is appended to the request so a long spec does not have to fit in Message. Existing `.py` names are locked on the inventory card before phase 0 so the committee continues the seed instead of starting empty. `.env`, venvs, `.git`, and binaries are not copied.

1.2.10: 2026-10-02

After coder smoke the host builds an implementation inventory (files plus top-level names/signatures) and merges it into one card: locked names from the first snapshot, this-round added/removed, and live names. Testers must call live names only (not invent APIs or copy the impl into the test); the host rejects tester writes and executes that call names missing from the live card. Later rounds keep locked names and only add; they do not wipe the card. Later tester turns re-execute only after a real SCORES line; an impl crash still lets the tester retarget to live names.

1.2.9: 2026-10-01

Host inventory (file list plus top-level defs) is shown to coder, evaluator, and manager. After critique, the evaluator scores items; the host then drops "missing file/function" claims that contradict inventory (and scores like discarded/0) and removes those rows from `next_fix` so the manager cannot copy them back. Manager phase 0 still writes the contract from the user prompt; later plan turns do not reread features/architecture and only append a new `next_fix` version from newest critiques plus a host done-list. On `impl_crash` (including `AttributeError: module ... has no attribute`), the coder's next write must be that impl file (module name, not the test traceback).

1.2.8: 2026-10-01

Coder gets the same inner loop as tester: write impl, host compile+import every non-test `.py` (does not run `if __name__=='__main__'`), fix stderr, repeat until the files load (cap 3). Quality is not checked at this step. Tester runs only after smoke passes. Coder may execute impl files; still cannot write tests. Qualify, vote, tester loop, and results/ `next_fix` are unchanged.

1.2.7: 2026-10-01

Host ownership rules stay on the board for every role (coder writes impl, tester writes/runs `test_*.py` only). The tester loop writes and executes until the harness prints n `SCORES` values or the crash is in impl (cap 3), then later rounds only re-execute. Critique sees a results packet, not JSONL; the host skips that LLM turn when there is no test file or the harness is broken, and merges `{owner,file,problem,fix}` into `next_fix` for the next coder/tester. `bench.py --tasks maze-path,csv-sql` runs a subset.

1.2.6: 2026-09-30

Host qualify no longer dies on a tester writing the impl file. A `tester_test_only` (or `coder_impl_only`) result is fed back and the tester turn continues so they can write/execute `test_*.py`. The same path is not executed six times in one turn; rewrite the test to run it again. If the tester never executes and a test file exists, the host runs it. Qualify uses the last `SCORES` line and the first `n` values (not a concatenation of every execute). Bracket lists like `kv_ok=[1,1,1,1]` parse; a single total when `n>=2` does not. Test writes without a `SCORES` print are rejected. If critique JSON is empty, the host writes critique items from stderr, denied writes, and the score gap. Coder cannot write test scripts.

1.2.5: 2026-09-30

Each coding run gets a folder under `coding-logs/` (not one mixed JSONL). Host appends `generic.jsonl` (role/tool events), `planner.jsonl` (phase 0 and each manager plan), `critique.jsonl`, and `test.jsonl` (test writes, execute stdout/stderr, parsed SCORES, qualify, whether a ballot actually ran). The same four files are mirrored into the work folder as `_logs/` (not listed as product files). Use these to see why a vote opened or why the next loop ignored a failed test.

1.2.4: 2026-09-30

Tester `sandbox_write` and `executeScriptInSandbox` are locked to test scripts (`test_*.py` or `*_test.py`). Writes to implementation files return `tester_test_only`. Coder still owns the rest of the work folder. This stops the tester from overwriting working code; qualify still needs the test file to print `SCORES`.

1.2.3: 2026-09-30

Coding committee host now lands sandbox files even when the model wraps JSON in markdown, truncates a `sandbox_write` payload, uses `file` /`script` instead of `path`, or nests `{"sandbox_write": {...}}` instead of `{"tool":...}`. Committee completions may use 1024 tokens (chat replies stay at 512). Truncated or fenced writes are applied once per role turn so the committee does not stall on repeated tool calls. `coding-logs/` showed the previous 0-file benches were parse/dispatch misses, not a broken write tool.

1.2.2: 2026-09-29

Coding committee writes a JSONL trace under `coding-logs/` on the stick (next to `logs/` and `outputs/`). Each role completion and each sandbox tool result is stored there, including `sandbox_write` ok/error. Chat History does not load these files. Setup creates the folder. It is not auto-purged; delete `coding-logs/` yourself when you want it gone.

1.2.1: 2026-09-28

Setup creates `logs/` and `outputs/` on the stick and stops if they are not writable. Missing **git** is a notice only. `REQUIREMENTS.md` lists the hard minimum. `README.pdf`, `REQUIREMENTS.pdf`, and `CHANGELOG.pdf` sit next to the Markdown with headings, bold, lists, and code styling (same files on Windows and Ubuntu). Setup and chat do not open them.

1.2.0: 2026-09-28

Coding committee, formatted History, and per-reply save buttons.

- **Coding** checkbox (always visible). Optional **GitHub URL** (`https://github.com/owner/repo` only) appears while Coding is on. The committee never starts unless that checkbox is on.
- Coding on: sequential roles on the **picked GGUF** (manager -> coder -> tester -> critique -> evaluator -> vote). Sandbox execute is tester-only. Clone only when a public GitHub URL is set and `git` exists on the PC. Work lands in `outputs/` on the stick. Status names the role; Stop still cancels between steps. History shows one finished reply, not each JSON pass.
- Coding off: the picked model still answers. Simple code requests keep a snippet plus "not tested." Asks to run or sandbox say it cannot test without Coding. High-stakes wording (legal / official / medical / ...) uses a short review pass with **no** sandbox.
- History paints Markdown as widgets (headings, lists, code boxes) via `formatter.py`. **View HTML** opens a sanitized page in the OS browser. **Save to notes** is on each Model reply (replaces Append last reply). **Save PDF** on a turn appears only if that turn wrote a PDF. Footer **Save transcript PDF** still exports the whole chat.
- Web Search is unchanged and still operator-gated.

1.1.3: 2026-09-26

If the loaded GGUF template rejects a `system` role, that text is folded into the first user message and the turn is retried. Fixes `(error: System role not supported)` on models such as Gemma, including the extra Web Search JSON passes.

1.1.2: 2026-09-26

While a reply or Web Search is running, the message box is locked and Send is replaced by a spinner. Stop stays available.

1.1.1: 2026-09-26

Web Search no longer dumps a whole page into one 4096-token window. HTML and page excerpts that will not fit (default 4096, minus the 512-token reply) are processed in parts and optionally summarized; only the last answer is shown in History.

1.1.0: 2026-09-26

Optional **Web Search** for the standalone chat (not Serp/Scraper, not the committee).

- Toggle above Message when the host is online; stays on until you turn it off. Asterisk hover warns that it is slow.
- On Send: model query -> Google HTML scrape (`search.py`) -> model remove-or-keep class/id pairs -> up to 3 URLs -> fetch and clean each page -> one answer in History.

- Intermediate steps (query, specs, URLs, page notes) go to `logs/web_YYYY-MM-DD_HHMMSS.jsonl`. Session turns can record `web_search` / `web_query` / `web_urls` / `web_log`.
- Setup also installs `requests` and `beautifulsoup4`.

1.0.11: 2026-09-26

The window is vertically scrollable when it is smaller than the full layout, so the message box padding still works outside fullscreen.

1.0.10: 2026-09-26

Looser page gutter (16px) and more space under the message box so the compose area is not flush against the window edge or the buttons.

1.0.9: 2026-09-26

When no older session log remains, History shows a one-time line that nothing is stored before this (or older logs were deleted).

1.0.8: 2026-09-26

Attach lines in history and logs use a document icon plus the **full path**. Older logs that only stored a name show the icon and that name.

1.0.7: 2026-09-26

shows *Reply to this message* on hover. Logs store the attach note (`shown` / `attached`); reloaded history paints `[attached name]` again.

1.0.6: 2026-09-26

History and Message are labeled separately. Empty history and a type-here hint. Send sits on the right of the box. Each You/Model line is bold with a user/bot mark and a control to quote that message. Clear quote is hidden until a reply is pending.

1.0.5: 2026-09-26

Setup also installs **pypdf** so attached PDFs can be read. If a library fails to install, the popup names that library (llama-cpp-python or pypdf) and says it is missing.

1.0.4: 2026-09-26

Setup popup now says the **llama-cpp-python** library is missing when that install fails.

1.0.3: 2026-09-26

Setup installs `llama-cpp-python` from the official CPU wheel index so Windows does not try to compile it (that failed without Visual Studio / nmake).

1.0.2: 2026-09-26

Setup leaves the system interpreter alone. Python 3.11+ builds or updates a host-only venv; older or missing Python shows a dialog (need 3.11+, found YY). Chat launchers start that venv.

1.0.1: 2026-09-26

Backup layout matches the USB `chat/` folder: scripts sit next to this README. Dropped the nested `chat/` directory and the factory `.env.example`.

1.0.0: 2026-09-26

First USB-only backup of the standalone chat.

- Chat UI, RAM-fit model picker, document attach, Reply/quote
- Session JSONL logs, 30-day purge, scroll-back as older context
- Confirmed `notes.md`, in-memory Save/Print PDF
- Host setup scripts (no venv on the stick)